Event Trading: Complete Setup Guide

🎯 Overview

Event trading captures **short-term volatility** around scheduled and unscheduled catalysts:

Event Type	Typical Move	Best Time to Trade	Win Rate
Earnings	3-10%	Pre or Post release	55-60%
FOMC	0.5-2%	Post-announcement fade	50-55%
NFP/CPI	0.3-1%	Post-release momentum	52-57%
Breaking News	1-5%	First 15 minutes	45-55%
Vol Spikes	2-5%	Continuation trade	48-53%

Key insight: Event trading is **higher risk, higher reward** than technical trading.

📋 Prerequisites

1. Get Free API Keys

Alpha Vantage (Earnings Calendar)

```bash
# Get free key: https://www.alphavantage.co/support/#api-key
# Free tier: 5 calls/minute, 500/day
export ALPHA_VANTAGE_API_KEY="your_key_here"
```

NewsAPI (Breaking News)

```bash
# Get free key: https://newsapi.org/register
# Free tier: 100 requests/day
export NEWS_API_KEY="your_key_here"
```

2. Update .env File

```bash
```

```
# Add to your .env
ALPHA_VANTAGE_API_KEY=demo
NEWS_API_KEY=
ENABLE_EVENT_TRADING=1
MAX_EVENT_POSITIONS=2
```

3. Install Dependencies

```bash
# Already in requirements.txt
pip install requests pytz pyyaml
```

🔧 Installation Steps

Step 1: Add Event System Files

```bash
# Create events directory
mkdir -p src/ib_quant_kit/events

# Add the files from artifacts:
# - event_detector.py
# - event_strategies.py
```

Step 2: Create Config File

Save the `event_config.yaml` artifact to your project root:

```bash
cp event_config.yaml ./event_config.yaml
```

Step 3: Update Main Loop

Add event trading to your main loop:

```python
```

```python
# In main_with_selection.py

from src.ib_quant_kit.events.event_detector import EventAggregator
from src.ib_quant_kit.events.event_strategies import (
    EventStrategyRouter,
    EventTradingManager
)

class TradingEngine:
    def __init__(self):
        # ... existing code ...

        # Add event trading components
        self.event_aggregator = EventAggregator()
        self.event_router = EventStrategyRouter()
        self.event_manager = EventTradingManager(
            self.event_aggregator,
            self.event_router
        )

        # Separate tracking for event vs technical trades
        self.event_positions = {}
        self.technical_positions = {}

    def run(self):
        while self.running:
            # ... existing technical trading ...

            # EVERY HOUR: Scan for events
            if now.minute == 0:  # Top of hour
                self._scan_and_trade_events()

            # Monitor active event trades
            self._manage_event_positions()

            time.sleep(10)

    def _scan_and_trade_events(self):
        """Scan for events and generate trade setups"""

        print("\n" + "="*60)
        print("EVENT SCAN")
        print("="*60)
```

```
    # Scan for events
    event_setups = self.event_manager.scan_and_generate_setups(
        symbols=settings.universe,
        market_data=self.market_data
    )

    if not event_setups:
        print("No actionable events found")
        return

    # Rank by priority
    ranked = self.event_manager.rank_setups(event_setups)

    # Check how many event positions we can take
    current_event_count = len(self.event_positions)
    max_event_positions = 3  # From config
    available_slots = max_event_positions - current_event_count

    if available_slots <= 0:
        print(f"Max event positions ({max_event_positions}) reached")
        return

    # Execute top setups
    for setup in ranked[:available_slots]:
        if self._execute_event_setup(setup):
            print(f"✅ Event trade entered: {setup.strategy_name}")

def _execute_event_setup(self, setup: EventTradeSetup) -> bool:
    """Execute an event trade setup"""

    # Circuit breaker check
    ok, reason = self.circuit_breaker.record_order()
    if not ok:
        print(f"Circuit breaker blocked event trade: {reason}")
        return False

    # Get contract
    data = self.market_data.get(setup.entry_symbol)
    if not data:
        return False

    contract = data['contract']
```

```
# Submit order
try:
    if setup.entry_type == "MARKET":
        # Market orders for immediate fills
        order_id = self.ib.submit_market(
            contract=contract,
            side=setup.entry_side,
            qty=setup.entry_qty
        )
    else:
        # Limit orders
        order_id = self.ib.submit_limit(
            contract=contract,
            side=setup.entry_side,
            qty=float(setup.entry_qty),
            limit=setup.entry_price,
            tif="DAY",
            outside_rth=False,
            idempotency_key=f"event-{setup.event.event_type.value}-{datetime.now().isoformat()}"
        )

    # Track as event trade
    self.event_positions[order_id] = {
        'setup': setup,
        'entry_time': datetime.now(),
        'entry_price': setup.entry_price,
        'symbol': setup.entry_symbol
    }

    print(f"📈 EVENT TRADE ENTERED:")
    print(f"   {setup.entry_side} {setup.entry_qty} {setup.entry_symbol} @ ${setup.entry_price:.2f}")
    print(f"   Strategy: {setup.strategy_name}")
    print(f"   Target: +{setup.target_pct:.1f}%, Stop: -{setup.stop_pct:.1f}%")
    print(f"   Max hold: {setup.time_exit_minutes} minutes")

    # Alert
    from src.ib_quant_kit.alerts.slack_alerts import send_slack
    send_slack(f"📈 Event Trade: {setup.strategy_name} - {setup.reasoning}")

    return True

except Exception as e:
    print(f"❌ Failed to execute event setup: {e}")
    return False
```

```python
def _manage_event_positions(self):
    """Monitor and exit event positions"""

    for order_id, event_pos in list(self.event_positions.items()):
        setup = event_pos['setup']
        entry_time = event_pos['entry_time']
        entry_price = event_pos['entry_price']
        symbol = event_pos['symbol']

        # Get current position
        pos = positions.get(symbol)
        if not pos or abs(pos.qty) < 0.01:
            # Position closed
            del self.event_positions[order_id]
            continue

        # Get current price
        current_price = self.market_data.get(symbol, {}).get('last', 0)
        if current_price <= 0:
            continue

        # Calculate hold time
        minutes_held = int((datetime.now() - entry_time).total_seconds() / 60)

        # Check exit conditions
        should_exit, reason = self.event_router.should_exit_event_trade(
            setup,
            current_price,
            entry_price,
            minutes_held
        )

        if should_exit:
            print(f"\n🔚 EXITING EVENT TRADE: {reason}")
            print(f"   {symbol}: held {minutes_held} min")
```